

"Build Smarter LLM Apps with Chains"

@abhi_techhub

Notes for a Smarter Tomorrow

LangChain

Complete Handbook

Build LLM Applications with ease

- ✓ Concepts
- ✓ Chains
- ✓ Tools & Agents
- ✓ RAG
- ✓ Real-world Examples
- ✓ Best Practices
- ✓ Interview Questions
- ✓ and Much More...

💡 Ideas
→ Chains
→ Real Impact



"Connect LLMs, Prompts and Tools to Build Real-World Applications"



Prompts
Better Inputs



LLMs
Smarter Responses



Tools
Extend Capabilities



Chains
Connect Components



Applications
Real-World Impact

"From Simple Chains to Powerful Applications"

- Learn
- Build
- Experiment
- Repeat
- Grow

"Better Tools
Better Products
Brighter Future"



Code Examples



Concepts & Notes



Real-World Use Cases

LangChain Today → Smarter AI Tomorrow

By
Abhishek Kumar
(@abhi_techhub)

"Building with LLMs just got easier!"

LangChain

Complete Handbook

Build LLM Applications from Idea to Production

@abhi_techhub

"Connect LLMs to the real world."

- ✓ Learn
- ✓ Build
- ✓ Experiment
- ✓ Apply

1 What is LangChain ?

- LangChain is an open-source framework to build applications with Large Language Models (LLMs).
- It provides modular components to connect LLMs with external data, tools and real-world systems.
- Helps in building complex LLM applications like RAG, Agents, Chatbots, Automation workflows etc.
- Supports multiple LLM providers (OpenAI, Anthropic, Google, etc.) and integrates with many tools and services.

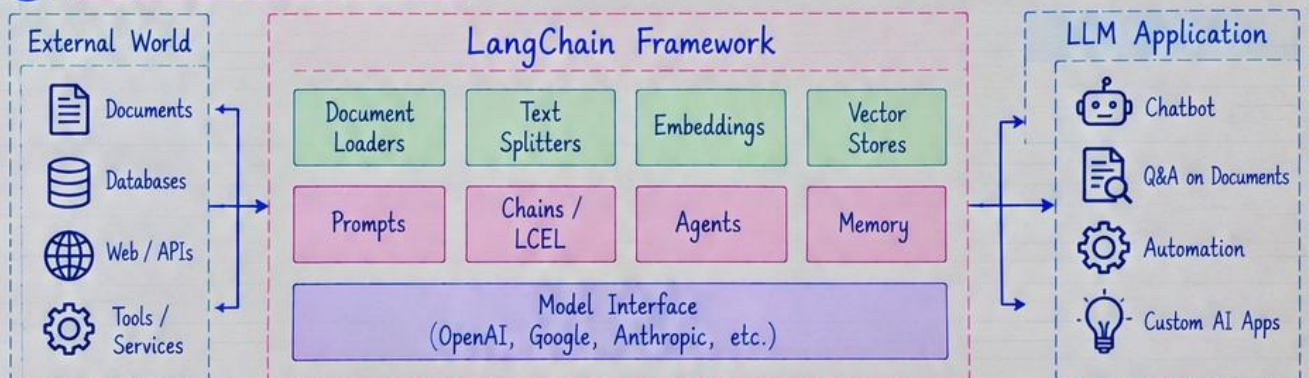
"LangChain makes it easy to go from LLM prototypes to production-ready applications."

2 Why LangChain ?

- LLMs alone are powerful but have limitations:
 - ✗ No real-time data.
 - ✗ No access to private/enterprise data.
 - ✗ Cannot take actions (e.g., call APIs, query databases).
 - ✗ Difficult to build complex workflows.
- LangChain solves these by providing:
 - ✓ Standardized interfaces to connect LLMs with data.
 - ✓ Tools, agents and memory for advanced applications.
 - ✓ Modular and composable components.
 - ✓ Active open-source community and ecosystem.

"LangChain bridges the gap between LLMs and real-world applications."

3 High Level Architecture



4 Key Features

- Modular Components - chains, agents, retrievers, memory, tools, etc.
- Wide Integrations - LLMs, vector databases, APIs, tools, data sources.
- Supports RAG - build retrieval augmented generation applications.
- Easy to Customize - create your own tools, chains and agents.
- Production Ready - works with LangSmith for debugging, tracing and monitoring.
- Large Community - lots of examples and resources.

5 LangChain vs Direct LLM API

Aspect	Direct LLM API	LangChain
Basic Usage	Simple prompts	Modular framework
External Data	Not supported	Supports (RAG)
Tool Usage	Manual implementation	Built-in tool support
Complex Workflows	Hard to manage	Easy (chains, agents)
Memory	Not available	Built-in memory modules
Integrations	Limited	Wide ecosystem
Production Ready	Needs custom setup	LangSmith, tracing, etc.

6 Popular Use Cases



"Same LLM. More Possibilities with LangChain."

Start small. Build something. Keep improving!



Ideas → Chains → Agents → Real Impact

[Page 01 of 15]

"Modular Building Blocks for LLM Apps"

LangChain Architecture

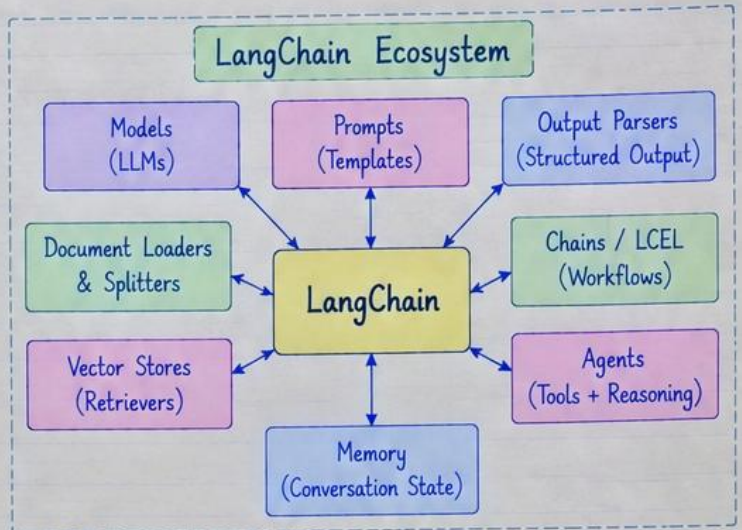
@abhi_techhub

"Same LLM. More Power with LangChain."

Understanding the Building Blocks

1 High Level Architecture

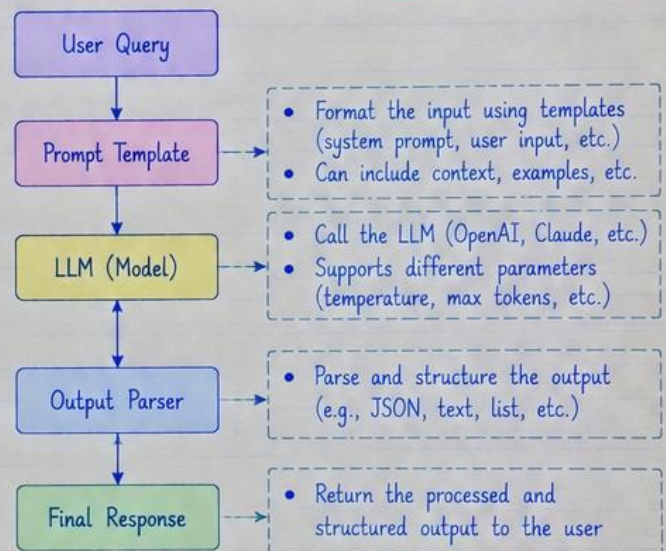
- LangChain provides a modular framework to build LLM applications by connecting different components together.
- It abstracts the complexity of working with LLMs, external data, tools and memory.
- You can use individual components or combine them to build complex workflows like RAG, Agents, Chatbots, etc.
- It follows a **composable and flexible** design.



2 Core Components

Component	Purpose
Models	Connect to LLM providers (OpenAI, Anthropic, Google, etc.)
Prompts	Manage and template prompts for better reusability
Chains / LCEL	Compose components into workflows
Tools	Allow LLMs to interact with external systems (APIs, databases, etc.)
Agents	Enable LLMs to reason, plan and use multiple tools
Memory	Maintain conversational context
Retrievers	Fetch relevant information from vector stores (RAG)
Document Loaders	Load data from PDFs, web pages, databases, etc.
Text Splitters	Split documents into smaller chunks
Vector Stores	Store embeddings for similarity search
Output Parsers	Convert LLM output into structured format (JSON, Pydantic, etc.)

3 How Components Work Together



4 Key Design Principles

- ✓ **Modularity** - use only what you need.
- ✓ **Composability** - combine components easily.
- ✓ **Provider Agnostic** - works with multiple LLMs.
- ✓ **Extensibility** - build custom tools, prompts, etc.
- ✓ **Production Ready** - supports tracing, evaluation, and deployment (e.g., LangSmith).

5 LangChain vs Other Frameworks

Feature	LangChain	LlamaIndex	Direct LLM API
Focus	General LLM Apps	Data/RAG focused	Basic LLM calls
Components	Wide (chains, agents, tools, memory, etc.)	Strong for data indexing & retrieval	None (build yourself)
Ease of use	High (modular)	High (RAG use cases)	Low (manual work)
Flexibility	Very high	High	Low
Best for	RAG, Agents, Chatbots, Production apps	Document search & knowledge apps	Simple experiments

"Understand the building blocks, then build without limits."

More Components → Bigger Possibilities



[Page 02 of 15]

"Same LLM.
More Possibilities
with LangChain."

LLM & Chat Models in LangChain

"Models are the brain,
LangChain gives
them superpowers."

Working with Different LLM Providers

1 What are LLMs & Chat Models ?

- LLMs (Large Language Models) are pre-trained models that understand and generate human-like text.
- Chat Models are LLMs optimized for conversational interactions (support roles: system, user, assistant).
- In LangChain, both are accessed through a unified interface, making it easy to switch between providers (OpenAI, Anthropic, Google, etc.).

"LangChain provides a consistent interface to work with different LLM providers."

2 Supported LLM Providers

Provider	LangChain Class	Popular Models
OpenAI	ChatOpenAI	gpt-4o, gpt-4, gpt-3.5
Anthropic	ChatAnthropic	claude-3, claude-3-haiku
Google	ChatGoogleGenerativeAI	gemini-pro, gemini-1.5
Meta (Local)	Ollama	llama3, mistral, etc.
HuggingFace	HuggingFaceEndpoint	open-source models
Cohere	ChatCohere	command-r, command-r+

3 Initialising a Chat Model

OpenAI

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.7,
    max_tokens=1000
)
```

You can change model, temperature, token limits and other parameters based on your use case.

Anthropic

```
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(
    model="claude-3-sonnet-20240229",
    temperature=0.5
)
```

Google (Gemini)

```
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-pro",
    temperature=0.7
)
```

4 Important Parameters

Parameter	Description
model	Specifies the model name (e.g., gpt-4o)
temperature	Controls randomness (0 = deterministic, 1 = more creative)
max_tokens	Maximum number of tokens to generate
top_p	Nucleus sampling (alternative to temperature)
streaming	Stream the response token by token
api_key	API key for the provider
base_url	Custom endpoint (useful for local models)

5 Simple Example

```
from langchain_openai import ChatOpenAI
from langchain.schema import HumanMessage, SystemMessage
llm = ChatOpenAI(model="gpt-4o", temperature=0.7)
messages = [
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="Explain LangChain in 2 lines.")
]
response = llm.invoke(messages)
print(response.content)
```

You can also use
.llm.invoke("prompt")
for simple use cases.

6 Chat vs LLM (Completion)

Chat Model	LLM (Completion Model)
Designed for conversations	Designed for text completion
Takes structured messages (system, user, assistant)	Takes a single prompt string
Better for chat applications	Useful for simple text generation
Examples: ChatOpenAI, ChatAnthropic, ChatGoogleGenerativeAI	Examples: OpenAI (text-davinci-003), HuggingFace models
Supports advanced features like function calling, tool use	Limited functionality

7 Key Takeaways

- LangChain provides a unified interface to work with multiple LLMs.
- Chat models are ideal for conversational use cases.
- You can easily switch between providers with minimal code changes.
- Tune parameters like temperature and max_tokens for better results.
- Use streaming for real-time responses.
- Experiment with different models to find the best fit for your use case.

"Different models. Same interface. Infinite Possibilities."



"A good prompt is like a good question - it unlocks better answers."

@abhi_techhub

Prompt Templates in LangChain

Design Better Prompts for Better Results

"Better Prompts. Smarter LLMs. Real Impact."

1 What are Prompt Templates ?

- Prompt templates help you create reusable and dynamic prompts for LLM applications.
- They allow you to insert variables (placeholders) into prompts and format them with actual values at runtime.
- LangChain provides different types of templates for different use cases (text, chat, few-shot, etc.).

"Prompt templates make it easy to manage, reuse and experiment with prompts."

2 Basic PromptTemplate (Text)

```
from langchain.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["topic"],
    template="""
    Explain the concept of {topic} in simple terms.
    Provide a short example.
    """
)

prompt = template.format(topic="Transformers")
print(prompt)
```

Use {variable} as placeholder. Pass variables using .format() or .invoke().

3 ChatPromptTemplate (For Chat Models)

```
from langchain.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI assistant."),
    ("user", "Explain {topic} in simple terms.")
])

messages = template.format_messages(topic="LangChain")
print(messages)
```

ChatPromptTemplate allows you to define a conversation with multiple roles (system, user, assistant).

4 Few-Shot Prompt Template

- Few-shot prompting provides examples in the prompt to guide the model's response.
- Useful for tasks like classification, Q&A, code generation, etc.

```
from langchain.prompts import FewShotPromptTemplate, PromptTemplate

examples = [
    {"input": "Happy", "output": "Positive"},
    {"input": "Terrible", "output": "Negative"}
]

example_template = "Input: {input}\nOutput: {output}"

prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate.from_template(example_template),
    suffix="Input: {text}\nOutput:",
    input_variables=["text"]
)
```

5 PromptTemplate with Partial Variables

- Use partial variables to fix some values in the template and provide the rest later.

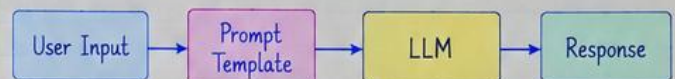
```
from langchain.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["name"],
    partial_variables={"company": "OpenAI"},
    template="""
    Hi {name},
    Welcome to {company}! How can I help you today?
    """
)

print(template.format(name="Abhishek"))
```

6 Prompt Chaining / Reusability

- Templates can be combined with LCEL or chains to build complex workflows.
- Keep your prompts modular and reusable.



You can also store templates in external files (e.g. .yaml, .json) for better maintainability.

7 Best Practices

- ✓ Be clear and specific in your instructions.
- ✓ Use system prompts to set the behavior.
- ✓ Provide examples for complex tasks (few-shot).
- ✓ Keep prompts concise but informative.
- ✓ Experiment with different phrasings.
- ✓ Use structured output (e.g. JSON) when needed.
- ✓ Test and iterate to get the best results.

"Good prompts lead to great conversations." 😊

8 Common Use Cases

- 📄 Summarization - Summarize long documents
- ❓ Q&A - Answer questions based on context
- </> Code Generation - Generate code from natural language
- 💬 Chatbots - Build conversational assistants
- 📋 Structured Output - Get JSON / structured responses
- ⚙️ Agents - Use in complex workflows with tools

Prompt → Test → Improve → Repeat

[Page 04 of 15]

"Small Building Blocks Big Possibilities."

@abhi_techhub

LangChain Expression Language (LCEL)

"Same Components. Different Flows. Endless Possibilities."

Compose LLM Applications Easily

A declarative way to build, chain and run LLM pipelines.

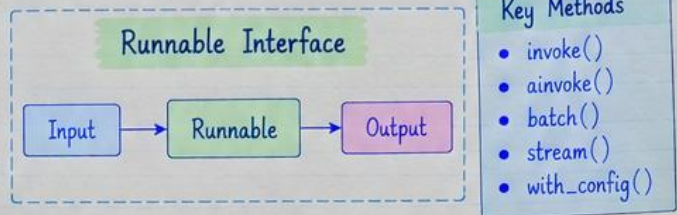
1 What is LCEL ?

- LCEL (LangChain Expression Language) is a simple and declarative way to compose LangChain components using a chainable syntax.
- It uses the `|` (pipe) operator to connect components.
- Each component is a Runnable (implements a common interface).
- It makes it easy to build complex LLM workflows without writing custom classes.

"LCEL turns individual components into powerful pipelines with minimal code."

2 Runnable Interface

- All LCEL components are Runnables.
- A Runnable takes input, processes it and returns output.
- Common methods: `invoke()`, `batch()`, `stream()`, `ainvoke()`.
- Examples of Runnables: `PromptTemplate`, `LLM`, `OutputParser`, `Retriever`, `Tool`, etc.



3 Basic LCEL Pipeline

A simple chain: Prompt → LLM → Output Parser

```
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate(
    template = """
    Explain {topic} in simple terms.
    """,
    input_variables = ["topic"]
)
```

Each component passes its output to the next using the `|` operator.

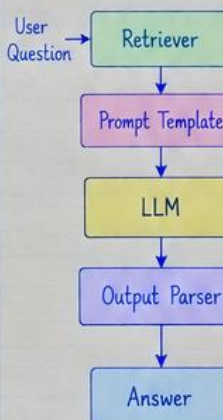
```
llm = ChatOpenAI(model="gpt-4o", temperature=0.7)
parser = StrOutputParser()
```

```
chain = prompt | llm | parser
```

```
result = chain.invoke({"topic": "LangChain"})
print(result)
```

4 More Complex Example

RAG Pipeline using LCEL



```
from langchain_core.runnables import RunnablePassthrough

rag_chain = (
    {
        "context": retriever,
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | parser
)

rag_chain.invoke("What is LCEL?")
```

LCEL makes it easy to pass multiple inputs using dictionaries and compose them together.

5 Useful LCEL Components

Component	Purpose
<code>RunnableSequence</code>	Chain multiple runnables in sequence (using <code> </code>)
<code>RunnableParallel</code>	Run multiple runnables in parallel
<code>RunnablePassthrough</code>	Pass inputs unchanged (useful in dicts)
<code>RunnableLambda</code>	Use a custom python function
<code>RunnableMap</code>	Apply a runnable to each item in a dict
<code>RunnableBranch</code>	Conditional execution
<code>RunnableWithFallbacks</code>	Fallback to another runnable on failure
<code>RunnableAssign</code>	Add new key-value pairs to the output
<code>RunnableRetry</code>	Retry on failure with custom logic

6 Example: Parallel & Custom Logic

RunnableParallel

```
from langchain_core.runnables import RunnableParallel

chain = RunnableParallel(
    summary = prompt1 | llm | parser,
    detailed = prompt2 | llm | parser
)

result = chain.invoke({"topic": "Transformers"})
print(result) # returns a dict with both outputs
```

Runs multiple chains in parallel and returns a dictionary of results.

RunnableLambda (Custom Function)

```
from langchain_core.runnables import RunnableLambda

def add_prefix(text):
    return "Final Answer: " + text

chain = prompt | llm | RunnableLambda(add_prefix) | parser
print(chain.invoke({"topic": "LCEL"}))
```

Use custom python functions inside your chain.

"Compose. Combine. Create. That's LCEL."

Simple Components → Powerful Applications



[Page 05 of 15]

"Chains turn building LLM apps from complex to simple."

Chains & Pipelines in LangChain

Combine Components to Build Powerful Workflows

"Small chains, Big possibilities."

1 What are Chains ?

- Chains are a way to combine multiple LangChain components (prompts, LLMs, tools, parsers, etc.) into a single workflow.
- They take input, pass it through a sequence of steps and return the final output.
- Chains provide a simple and reusable way to build LLM applications.
- In modern LangChain, chains are built using LCEL (RunnableSequence) instead of older Chain classes.

"A chain is a pipeline of components that work together to achieve a goal."

2 Simple Chain Example

Prompt → LLM → Output Parser

```
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

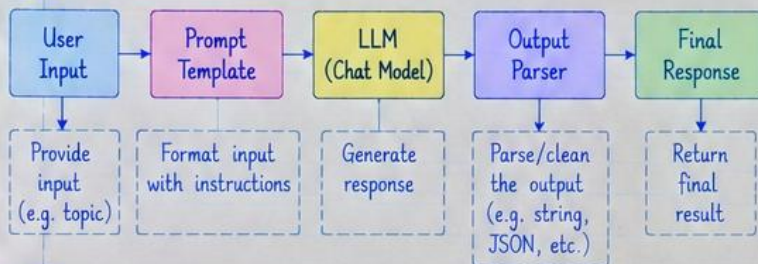
# Define components
prompt = PromptTemplate(
    template="Explain {topic} in simple terms.",
    input_variables=["topic"]
)
llm = ChatOpenAI(model="gpt-4o", temperature=0.7)
parser = StrOutputParser()

# Create chain using | (LCEL)
chain = prompt | llm | parser

# Run the chain
result = chain.invoke({"topic": "LangChain"})
print(result)
```

The | operator connects components and creates a RunnableSequence behind the scenes.

3 Chain Flow Diagram



4 Common Types of Chains

Chain Type	Description	Example Use Case
LLMChain	Basic chain with prompt + LLM	Simple text generation
Sequential Chain	Run multiple chains in sequence	Summarize → translate
Router Chain	Route to different chains based on input	Classify and process accordingly
Transform Chain	Transform input before passing to LLM	Clean/format input
Retrieval Chain	Combine with retrievers (RAG)	Question answering on documents
Conversation Chain	Maintain chat history	Chatbots

6 Best Practices

- ✓ Keep chains modular and reusable.
- ✓ Use meaningful prompts with clear instructions.
- ✓ Use output parsers for structured output (JSON, lists, etc.).
- ✓ Handle errors and add retries where needed.
- ✓ For complex workflows, use LCEL or LangGraph.
- ✓ Log intermediate outputs for debugging.
- ✓ Combine with memory, tools or retrievers as per use case.

"Chains make ideas executable."



Plan → Chain → Execute → Scale

5 Sequential Chain Example

```
from langchain.chains import SequentialChain, LLMChain
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4o", temperature=0.7)

# Chain 1: Generate a summary
summary_prompt = PromptTemplate(
    input_variables=["text"],
    template="Summarize the following text: {text}"
)
chain1 = LLMChain(llm=llm, prompt=summary_prompt, output_key="summary")
```

```
# Chain 2: Translate the summary
translate_prompt = PromptTemplate(
    input_variables=["summary"],
    template="Translate this to Hindi: {summary}"
)
chain2 = LLMChain(llm=llm, prompt=translate_prompt, output_key="translation")
```

```
# Combine chains
overall_chain = SequentialChain(
    chains=[chain1, chain2],
    input_variables=["text"],
    output_variables=["summary", "translation"]
)
result = overall_chain.invoke({"text": "LangChain is awesome!"})
print(result)
```

SequentialChain runs multiple chains one after another, passing the output of one as input to the next.

"Your data can be anywhere, LangChain helps you bring it together."

Document Loading & Text Splitting in LangChain

@abhi_techhub

"Good data preparation leads to better answers."

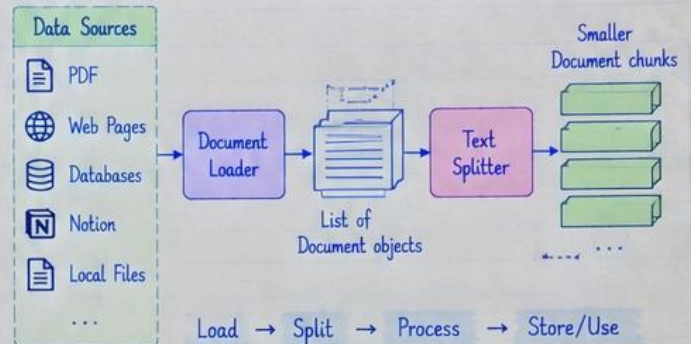
Prepare Your Data for LLM Applications

1 Why Document Loading is Important?

- Real world data is available in different formats (PDFs, websites, databases, Notion, etc.).
- LLMs can't directly work with raw files.
- Document loaders help you load data into a standard format (Document objects).
- Text splitting breaks large documents into smaller chunks for better processing, embedding and retrieval.

"Loading and splitting documents is the first step in building RAG applications."

2 Document Loader Architecture



3 Common Document Loaders

Loader	Use Case	Import
PyPDFLoader	Load PDF files	<code>from langchain_community.document_loaders import PyPDFLoader</code>
WebBaseLoader	Load web pages (URLs)	<code>from langchain_community.document_loaders import WebBaseLoader</code>
DirectoryLoader	Load all files from a directory	<code>from langchain_community.document_loaders import DirectoryLoader</code>
CSVLoader	Load CSV files	<code>from langchain_community.document_loaders import CSVLoader</code>
UnstructuredLoader	Load various file formats (docx, pptx, etc.)	<code>from langchain_community.document_loaders import UnstructuredLoader</code>
NotionDirectoryLoader	Load Notion pages	<code>from langchain_community.document_loaders import NotionDirectoryLoader</code>
SQLDatabaseLoader	Load data from SQL DB	<code>from langchain_community.document_loaders import SQLDatabaseLoader</code>
S3Loader	Load files from AWS S3	<code>from langchain_community.document_loaders import S3DirectoryLoader</code>

4 Example: Load a PDF

```

from langchain_community.document_loaders import
import PyPDFLoader

loader = PyPDFLoader("example.pdf")
docs = loader.load()

print(f"Loaded {len(docs)} pages")
print(docs[0].page_content[:200])

```

Each document is a Document object with:

- page_content → actual text
- metadata → metadata (page number, source, etc.)
- id → unique identifier

5 Text Splitting

- Large documents are split into smaller chunks.
- Helps in better embeddings and retrieval.
- Overlap is used to maintain context between chunks.
- LangChain provides multiple text splitters.

"Good chunking = better retrieval = better answers."

6 Common Text Splitters

Splitter	Use Case	Import
CharacterTextSplitter	Simple character based split	<code>from langchain.text_splitter import CharacterTextSplitter</code>
RecursiveCharacterTextSplitter	Smart split (recommended)	<code>from langchain.text_splitter import RecursiveCharacterTextSplitter</code>
TokenTextSplitter	Split based on tokens	<code>from langchain.text_splitter import TokenTextSplitter</code>
MarkdownHeaderSplitter	Split by markdown headers	<code>from langchain.text_splitter import MarkdownHeaderTextSplitter</code>
HTMLHeaderTextSplitter	Split HTML content	<code>from langchain.text_splitter import HTMLHeaderTextSplitter</code>

7 Example: Split Documents

```

from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)

chunks = splitter.split_documents(docs)

print(f"Total chunks: {len(chunks)}")
print(chunks[0].page_content[:200])

```

chunk_size → size of each chunk
chunk_overlap → overlap between chunks (helps maintain context)

8 Best Practices

- ✓ Choose the right loader for your data source.
- ✓ Use RecursiveCharacterTextSplitter for most cases.
- ✓ Experiment with chunk_size (500 - 2000) and overlap (50 - 300).
- ✓ Keep important context together (headings, sections).
- ✓ Clean and preprocess data if needed (remove noise).
- ✓ Validate chunks before embedding.
- ✓ Use metadata (source, page number, etc.) for better traceability.

"Load. Split. Embed. Retrieve. Answer."



Better Data → Better Retrieval → Better LLM Apps

[Page 07 of 15]

"Convert your data into knowledge for your LLM."

Embeddings & Vector Stores in LangChain

@abhi_techhub

"Similar meaning lives closer in vector space."

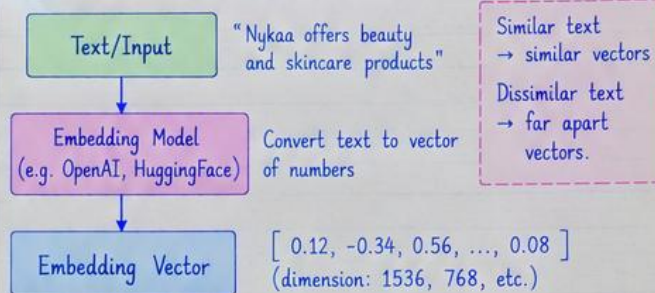
Store and Retrieve Knowledge Efficiently

1 What are Embeddings ?

- Embeddings convert text (or other data) into high-dimensional vectors of numbers.
- They capture the semantic meaning of the text.
- Similar pieces of text have similar embeddings (closer vectors) in the vector space.
- Used for similarity search, clustering, classification, RAG, etc.

"Embeddings help machines understand the meaning, not just the words."

2 How Embeddings Work ?



3 Embedding Models in LangChain

Provider	Class	Model Name	Dimension
OpenAI	OpenAIEmbeddings	text-embedding-3-small	1536
		text-embedding-3-large	3072
HuggingFace	HuggingFaceEmbeddings	all-MiniLM-L6-v2	384
		BAAI/bge-base-en	768
Cohere	CohereEmbeddings	embed-english-v3.0	1024
Google	GoogleGenerativeAIEmbeddings	models/embedding-001	768

Choose the model based on your use case, language support, performance and cost.

4 Example: Create Embeddings

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

text = "LangChain makes it easy to build LLM apps."
vector = embeddings.embed_query(text)

print(len(vector)) # 1536
print(vector[:5]) # first 5 dimensions
```

You can also use `embed_documents()` for multiple texts at once.

6 Popular Vector Stores in LangChain

Vector Store	LangChain Class	Use Case
FAISS	FAISS	Local, fast, easy to use
Chroma	Chroma	Local, persistent, developer friendly
Pinecone	Pinecone	Managed cloud service
Weaviate	Weaviate	Open source, scalable
Milvus	Milvus	High performance, large scale
Qdrant	Qdrant	Open source, high performance
Astra DB	AstraDB	Serverless vector DB (DataStax)

5 What are Vector Stores ?

- Vector stores store embeddings and allow fast similarity search.
- They help retrieve relevant documents for a given query based on vector similarity (e.g. cosine similarity).
- Essential for RAG applications.

"Vector stores are like a searchable memory for your LLM."

7 Example: Using FAISS (Local)

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load and split documents
loader = TextLoader("notes.txt")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
                                          chunk_overlap=100)
documents = splitter.split_documents(docs)

# Create embeddings and store in FAISS
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = FAISS.from_documents(documents, embeddings)

# Save locally
vectorstore.save_local("faiss_index")
```

You can also load existing index using `FAISS.load_local()`

8 Similarity Search Example

```
query = "best skincare products for dry skin"
results = vectorstore.similarity_search(query, k=3)
for i, doc in enumerate(results):
    print(f"Result {i+1}:", doc.page_content[:200])
```

Other useful methods:

- `similarity_search_with_score()` → also returns similarity score
- `max_marginal_relevance_search()` → diverse results
- `as_retriever()` → use with RAG chains

9 Key Takeaways

- ✓ Embeddings capture semantic meaning of text.
- ✓ Vector stores enable fast similarity search.
- ✓ Choose the right embedding model and vector store for your use case.
- ✓ Essential building block for RAG applications.
- ✓ Experiment with different models and chunk sizes for better results.

"Good embeddings + right retrieval = powerful LLM applications."



"Your documents can be the superpower for your LLM."

RAG with LangChain

Build Real-World Knowledge Based Applications

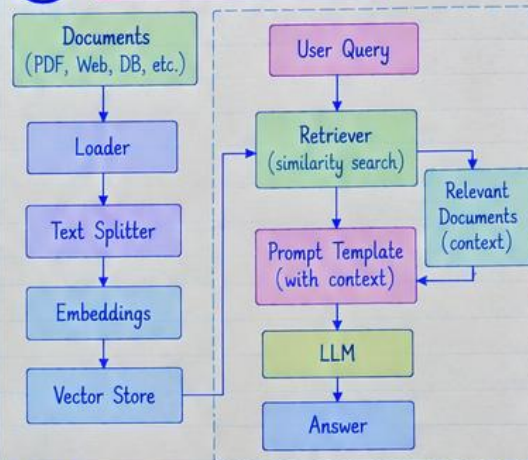
"Right context leads to right answers."

1 What is RAG ?

- RAG (Retrieval Augmented Generation) combines information retrieval with LLM generation.
- Instead of relying only on the model's internal knowledge, it retrieves relevant information from external sources (your documents, databases, etc.).
- This helps in generating accurate, up-to-date and context-aware responses.

"RAG makes LLMs more factual, reliable and useful for real-world applications."

2 RAG Architecture



Key Idea:
Retrieve relevant context before generating the answer → better accuracy!

3 Step 1: Load Documents

LangChain provides multiple document loaders.

Loader	Use Case	Example
PyPDFLoader	PDF files	PyPDFLoader("file.pdf")
WebBaseLoader	Web pages	WebBaseLoader("https://...")
DirectoryLoader	Multiple files (folder)	DirectoryLoader("/docs")
NotionLoader	Notion pages	NotionLoader(...)
CSVLoader	CSV files	CSVLoader("data.csv")
TextLoader	Plain text files	TextLoader("notes.txt")

4 Step 2: Split Documents

- Documents are split into smaller chunks to fit into LLM context window and for better retrieval.
- Common splitters: CharacterTextSplitter, RecursiveCharacterTextSplitter, TokenTextSplitter.

```

from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
docs = splitter.split_documents(documents)
  
```

Tip:
Smaller chunks → better retrieval but too small may lose context. (Experiment!)

5 Step 3: Create Embeddings

- Embeddings convert text into high-dimensional vectors that capture semantic meaning.
- Popular embedding models: OpenAI, HuggingFace, Cohere, etc.

```

from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
  
```

Embedding model converts text chunks into vectors that can be stored in a vector database.

6 Step 4: Store in Vector Store

- Vector store stores the embeddings and enables fast similarity search.
- Popular vector stores: FAISS, Chroma, Pinecone, Weaviate, Milvus, Qdrant, etc.

```

from langchain_community.vectorstores import FAISS
vectorstore = FAISS.from_documents(
    documents=docs,
    embedding=embeddings
)
  
```

You can also use Chroma (local & easy to set up) for small projects.

7 Step 5: Retrieve & Generate

- Retrieve relevant documents for the user's query and pass them to the LLM with a proper prompt.
- Use retrievers like similarity_search or as_retriever().

```

retriever = vectorstore.as_retriever(search_type="similarity",
                                     search_kwargs={"k": 4})

from langchain_chains import RetrievalQA
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o", temperature=0.2)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever
)
result = qa_chain.invoke({"query": "What is RAG?"})
print(result["result"])
  
```

Retriever finds the most relevant chunks which are added to the prompt before generation.

8 Use Cases of RAG

- ✓ Chat with your PDF documents (reports, research papers).
- ✓ Internal knowledge base for companies.
- ✓ Customer support using product documentation.
- ✓ Q&A on websites, wikis, Notion, Confluence.
- ✓ Legal, medical, finance document analysis.
- ✓ Personalized learning assistants.



Key Takeaways

- ✓ RAG reduces hallucinations.
- ✓ Good document splitting and embeddings are crucial.
- ✓ Choose the right vector store for your scale.
- ✓ Always use a well-designed prompt with retrieved context.
- ✓ Experiment with chunk size, k value and retrieval method.
- ✓ RAG + Agents = even more powerful applications!

"Ground your LLM with real data."



"Agents turn LLMs from responders into doers."

Agents in LangChain

Give LLMs the ability to use tools, take actions and achieve goals

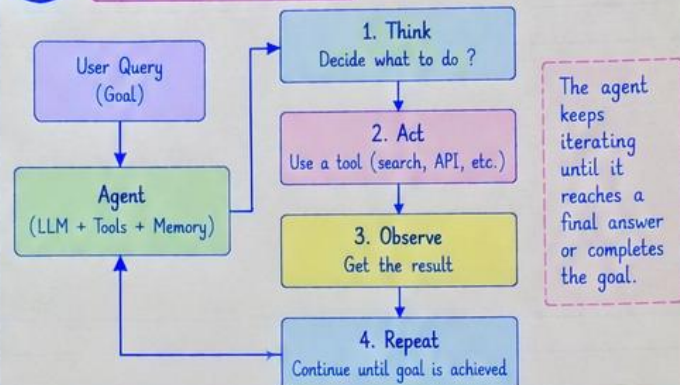
"More than answers, Agents take action."

1 What are Agents ?

- Agents are LLMs that can use tools, make decisions and take actions to achieve a goal.
- They follow a loop of: Think → Act → Observe → Repeat
- Agents can interact with external systems (APIs, databases, search, etc.)
- LangChain provides easy abstractions to build agents with tools, memory and custom logic.

"An agent is an LLM that can plan, use tools, and learn from results to accomplish a task."

2 Agent Workflow



3 Types of Agents in LangChain

Agent Type	Description	Use Case
Zero-Shot Agent	Uses tool description to decide actions	General purpose tasks
ReAct Agent	Uses reasoning + actions (Think → Act → Observe)	Complex, multi-step tasks
Conversational Agent	Maintains chat history with memory	Chatbots, assistants
Structured Chat Agent	Works with structured input/output	APIs, data tasks
OpenAI Functions Agent	Uses function calling	Reliable tool calling (with OpenAI models)

4 Example: Zero-Shot Agent with Tools

```

from langchain.agents import initialize_agent, AgentType
from langchain.tools import DuckDuckGoSearchRun, PythonREPLTool
from langchain_openai import ChatOpenAI

# Define tools
search = DuckDuckGoSearchRun()
repl = PythonREPLTool()
tools = [search, repl]

# Initialize LLM
llm = ChatOpenAI(model="gpt-4o", temperature=0)

# Create agent
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

# Run agent
result = agent.invoke({"input": "What is the population of India and plot a bar chart of top 5 states using python?"})
print(result)
  
```

This agent can:

- Search the web
- Run Python code
- Combine results
- Give final answer

5 Tools in Agents

- Tools allow agents to interact with external systems.
- Common tools: Search (DuckDuckGo, SerpAPI), Python REPL, APIs, Databases, File system, etc.
- You can also create your own custom tools.

Example: Custom Tool

```

from langchain.tools import tool
@tool
def get_weather(city: str) -> str:
    """Get current weather for a city."""
    # in real app, call a weather API
    return f"The weather in {city} is 28°C and sunny."
  
```

Custom tools help agents work with your own APIs and business logic.

6 Memory in Agents

- Memory helps agents remember past conversations and make better decisions.
- LangChain provides different memory types:
 - ConversationBufferMemory - stores full conversation
 - ConversationSummaryMemory - stores summary
 - ConversationBufferWindowMemory - keeps last N messages
 - VectorStoreRetrieverMemory - uses vector store for long-term

"Memory makes agents more contextual and useful over multiple turns."

7 Real-World Use Cases

- ✓ Research assistant (search, summarize, analyze)
- ✓ Data analysis (query DB, run code, generate reports)
- ✓ Customer support chatbot (FAQs, order status, etc.)
- ✓ Travel planner (search flights, hotels, create itinerary)
- ✓ Financial analysis (fetch data, analyze, summarize)
- ✓ Automate business workflows (CRM, email, slack, etc.)

8 Key Takeaways

- ✓ Agents can use tools to take real actions.
- ✓ Choose the right agent type for your use case.
- ✓ Provide clear tool descriptions for better performance.
- ✓ Use memory for conversational and long-term tasks.
- ✓ Start simple, then add more tools and capabilities.
- ✓ Agents + Tools + Memory = Powerful Real-World Applications.

"From questions to actions - that's the power of agents."



"A good memory makes AI feel human."

Memory in LangChain

Give your LLMs the ability to remember

@abhi_techhub

"Same user. Better answers. That's memory."

1 What is Memory ?

- Memory allows LLMs to remember past conversations and context across multiple turns.
- By default, LLMs are stateless (they don't remember previous messages).
- Memory helps in building more contextual, personalized and intelligent applications.
- LangChain provides different types of memory based on use case and storage needs.

"Memory turns a single conversation into a meaningful experience."

2 Why Memory is Important ?

- Provides context for follow-up questions.
- Improves user experience in chat applications.
- Essential for chatbots, virtual assistants and support systems.
- Helps maintain long-term context (user preferences, history).
- Useful for multi-turn conversations and complex tasks.

Example:

User: My name is Abhishek.

AI: Nice to meet you!

User: What is my name?

AI (with memory): Your name is Abhishek.

AI (without memory): I don't have that information.

Without memory, every conversation feels like the first one.

3 Common Memory Types

Memory Type	Description	Use Case
ConversationBufferMemory	Stores all messages in memory	Simple chatbots
ConversationSummaryMemory	Stores a summary of conversations	Long conversations
ConversationBufferWindowMemory	Keeps last N messages	Limit context size
EntityMemory	Remembers key entities (people, places, etc.)	Personalized bots
VectorStoreRetrieverMemory	Stores long-term memory in a vector store	Long-term knowledge
ConversationKnowledgeGraphMemory	Builds a knowledge graph from conversations	Complex, relational data

4 Example: ConversationBufferMemory

```
from langchain.memory import ConversationBufferMemory
from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationChain
```

Initialize memory

```
memory = ConversationBufferMemory()
```

```
llm = ChatOpenAI(model="gpt-4o", temperature=0.7)
```

```
chain = ConversationChain(
```

```
    llm=llm,
    memory=memory
)
```

Run conversation

```
print(chain.predict(input="Hi, my name is Abhishek."))
```

```
print(chain.predict(input="What is my name?"))
```

The model remembers your previous messages using memory.

5 Example: ConversationSummaryMemory

```
from langchain.memory import ConversationSummaryMemory
from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationChain
```

```
memory = ConversationSummaryMemory(
    llm=ChatOpenAI(model="gpt-4o", temperature=0.7)
)
```

```
chain = ConversationChain(
    llm=ChatOpenAI(model="gpt-4o", temperature=0.7),
    memory=memory
)
```

```
print(chain.predict(input="Tell me about LangChain in simple terms."))
```

Useful for long chats - stores a summary instead of full history.

6 Example: VectorStoreRetrieverMemory

```
from langchain.memory import VectorStoreRetrieverMemory
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
```

```
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_texts([], embeddings)
```

```
memory = VectorStoreRetrieverMemory(
    retriever=vectorstore.as_retriever(),
    memory_key="chat_history"
)
```

```
print("Vector store memory initialized!")
```

Stores conversations in a vector DB for long-term memory and semantic search.

7 When to Use Which Memory ?

Use Case	Recommended Memory
Short conversations	ConversationBufferMemory
Long conversations	ConversationSummaryMemory
Limit memory size	ConversationBufferWindowMemory
Remember key facts/entities	EntityMemory
Persist knowledge long term	VectorStoreRetrieverMemory
Complex multi-entity data	ConversationKnowledgeGraphMemory

8 Best Practices

- ✓ Choose the right memory type for your use case.
- ✓ Be mindful of token limits (summarize or use window memory).
- ✓ Use vector store memory for long-term knowledge.
- ✓ Combine memory with retrievers for better results (RAG).
- ✓ Sanitize and manage sensitive user data.
- ✓ Persist memory if needed (database or vector store).
- ✓ Test memory behavior with real-world conversations.
- ✓ Keep the conversation relevant and avoid context pollution.

"Memory today, smarter conversations tomorrow."



"Chains turn components into real applications."

Chains in LangChain

Connect LLMs, Prompts and Tools to Build Workflows

@abhi_techhub

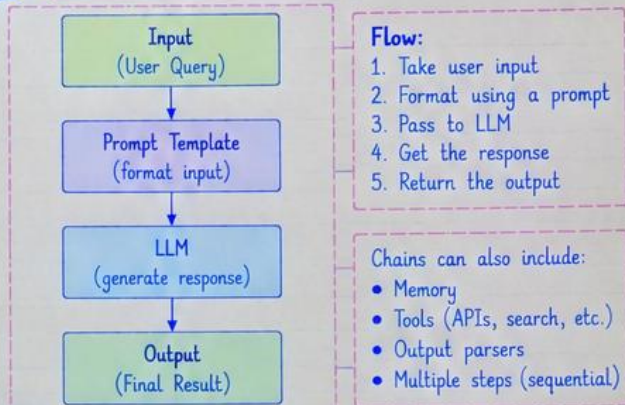
"Small chains today, Powerful apps tomorrow."

1 What are Chains ?

- Chains in LangChain are sequences of components (LLMs, prompts, tools, etc.) that work together to perform a task.
- They allow us to create multi-step workflows in a simple and modular way.
- Instead of calling each component manually, chains handle the flow of data automatically.
- Useful for complex tasks like RAG, summarization, chatbots, data extraction, etc.

"Chains make it easy to go from a simple LLM call to a full application."

2 How Chains Work ?



3 Types of Chains

Chain Type	Description	Use Case
LLMChain	Basic chain with prompt + LLM	Simple Q&A, text generation
SequentialChain	Run multiple chains in sequence	Multi-step workflows
RouterChain	Route input to different chains	Handle different queries
RetrievalChain	Combine retriever with LLM (RAG)	Question answering with external data
ConversationalChain	Adds memory for chat	Chatbots, assistants
TransformChain	Custom logic between steps	Data processing, custom tasks

4 Example: Simple LLMChain

```
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

# Define prompt
template = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple terms."
)
# Initialize LLM
llm = ChatOpenAI(model="gpt-4o", temperature=0.7)
# Create chain
chain = LLMChain(llm=llm, prompt=template)
# Run chain
result = chain.invoke({"topic": "Kubernetes"})
print(result["text"])
```

This is the most basic chain. You can replace ChatOpenAI with any LLM (Gemini, Claude, Llama, etc.).

5 Example: SequentialChain

```
from langchain.chains import LLMChain, SequentialChain
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0.7)
prompt1 = PromptTemplate(
    input_variables=["topic"],
    template="Write a short note on {topic}."
)
prompt2 = PromptTemplate(
    input_variables=["note"],
    template="Summarize the following in 3 bullet points: {note}"
)
chain1 = LLMChain(llm=llm, prompt=prompt1, output_key="note")
chain2 = LLMChain(llm=llm, prompt=prompt2, output_key="summary")
sequential_chain = SequentialChain(
    chains=[chain1, chain2],
    input_variables=["topic"],
    output_variables=["note", "summary"]
)
result = sequential_chain.invoke({"topic": "Docker"})
print(result["summary"])
```

SequentialChain runs multiple chains one after another, passing the output of one as input to the next.

6 Example: RetrievalChain (RAG)

```
from langchain.chains import RetrievalQA
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings, ChatOpenAI

# Load documents and create vector store
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(documents, embeddings)
retriever = vectorstore.as_retriever(search_type="similarity")

# Create chain
chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o"),
    retriever=retriever,
    chain_type="stuff"
)
result = chain.invoke({"query": "What is RAG?"})
print(result["result"])
```

RetrievalChain combines a retriever (vector store) with an LLM to answer questions using your own documents.

7 When to Use Chains ?

- ✓ When your task involves multiple steps.
- ✓ When you need to combine LLMs with tools, memory or retrievers.
- ✓ When you want to build reusable and modular workflows.
- ✓ Useful for real-world applications (RAG, chatbots, automation, etc.).

8 Key Takeaways

- ✓ Chains simplify complex LLM workflows.
- ✓ Choose the right chain type for your use case.
- ✓ Combine with memory, tools and retrievers for powerful apps.
- ✓ Start simple, then gradually build complex chains.
- ✓ Chains are the backbone of many real-world LangChain applications.

"Chains connect ideas to intelligent actions."



"LLMs are powerful, but tools give them superpowers."

Tools in LangChain

Extend LLMs with external capabilities

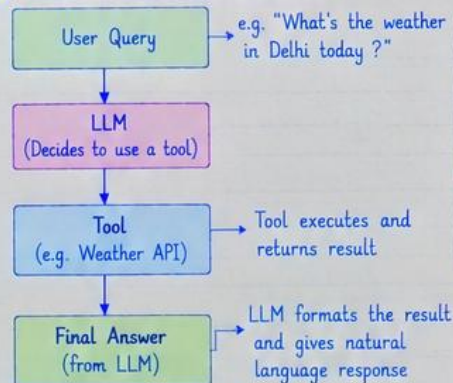
@abhi_techhub
"Give your LLM access to the real world."

1 What are Tools ?

- Tools allow LLMs to interact with external systems and perform real actions like searching the web, running code, querying databases, calling APIs, etc.
- In LangChain, tools are Python functions or wrappers that the LLM can call when needed.
- LLM decides when to use a tool based on the user query and the tool description.
- Tools help LLMs go beyond text generation and solve real-world problems.

"LLMs generate intelligence, Tools bring it into the real world."

2 How Tools Work ?



Key point:
LLM chooses the right tool based on the tool description and the user query.

3 Types of Tools in LangChain

Tool Type	Description	Example	Use Case
Built-in Tools	Ready to use tools from LangChain	SerpAPI, DuckDuckGo	Web search
Custom Tools	User-defined functions	Python function	Custom logic
API Tools	Call external APIs	REST APIs	Real-time data
Database Tools	Query databases	SQLDatabase	Enterprise data
Code Execution Tools	Run Python code	PythonREPLTool	Data analysis
File System Tools	Read/write files	FileManagementTool	Document Q&A

4 Example: Using Built-in Tool (SerpAPI)

```
from langchain.tools import SerpAPIWrapper
from langchain.agents import initialize_agent, AgentType
from langchain_openai import ChatOpenAI

# Initialize tool
search = SerpAPIWrapper()
# Initialize LLM
llm = ChatOpenAI(model="gpt-4o", temperature=0)
# Create agent with search tool
agent = initialize_agent(
    tools=[search],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)
result = agent.invoke({"input": "Search latest news about ISRO."})
print(result["output"])
```

The LLM will use the search tool to get real-time information from the web.

5 Example: Custom Tool

```
from langchain.tools import tool

@tool
def calculate_bmi(weight: float, height: float) -> str:
    """Calculate BMI from weight (kg) and height (m)."""
    bmi = weight / (height ** 2)
    return f"Your BMI is {bmi:.2f}"

# Use in agent
from langchain.agents import initialize_agent, AgentType
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0)
agent = initialize_agent(
    tools=[calculate_bmi],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)
result = agent.invoke({"input": "My weight is 70 and height is 1.75. What is my BMI?"})
print(result["output"])
```

Use @tool decorator to convert any Python function into a LangChain tool.

6 Example: Using Multiple Tools

```
from langchain.tools import SerpAPIWrapper, PythonREPLTool
from langchain.agents import initialize_agent, AgentType
from langchain_openai import ChatOpenAI

search = SerpAPIWrapper()
python_repl = PythonREPLTool()
llm = ChatOpenAI(model="gpt-4o", temperature=0)
agent = initialize_agent(
    tools=[search, python_repl],
    llm=llm,
    agent=AgentType.OPENAI_FUNCTIONS,
    verbose=True
)
result = agent.invoke({"input": "Search population of India and calculate 5% of it using Python."})
print(result["output"])
```

Agents can use multiple tools and decide which one to use for each step.

7 Best Practices

- ✓ Give clear and descriptive names and docstrings for tools.
- ✓ Use the right tool for the right use case.
- ✓ Handle errors and edge cases in custom tools.
- ✓ Limit the number of tools to avoid confusion.
- ✓ Combine tools with memory for better multi-turn conversations.
- ✓ Test tools independently before using in agents.

8 Key Takeaways

- ✓ Tools extend LLM capabilities beyond text.
- ✓ LangChain provides many built-in tools (search, APIs, DB, etc.).
- ✓ You can easily create custom tools using @tool.
- ✓ Agents can use multiple tools to solve complex tasks.
- ✓ Tools + Agents + Memory = powerful real-world applications.
- ✓ Always choose the right tool and provide clear descriptions.

"The right tool at the right time turns an LLM into a problem solver."



"Better Retrieval Leads to Smarter Answers."

RAG Advanced Concepts

Improve Retrieval, Reasoning and Real-World Performance

@abhi_techhub

"From simple search to intelligent knowledge systems."

1 Why Advanced RAG ?

- Basic RAG works well for simple use cases, but real-world applications need more.
- Advanced RAG techniques improve:
 - Retrieval quality (better context)
 - Answer accuracy (less hallucinations)
 - Handling large and complex data
 - Domain-specific applications (legal, medical, etc.)

"Advanced RAG = Better Retrieval + Smarter Reasoning + Reliable Answers."

3 Advanced Retrieval Techniques

Technique	What it does ?	Use Case
Query Expansion	Expands user query with similar terms	Improves recall
Hybrid Search (Dense + Sparse)	Combines vector search and keyword search (BM25)	Better relevance
Reranking	Re-ranks top results using a cross-encoder	Improves precision
Metadata Filtering	Filters results using metadata (date, source, etc.)	Domain-specific search
Multi-Query Retrieval	Generates multiple query variations	Complex questions
Context Compression	Summarizes retrieved documents	Fit more context
Self-Querying	LLM converts query to structured filters	Structured data search
Hierarchical Retrieval	Search across document structure (sections, headings)	Large documents

5 Context Compression

- Large documents may exceed context window.
- Use compression to keep only the most relevant information.
- Techniques:
 - LLM summarization
 - Extractive summarization (e.g. TextRank)
 - Recursive summarization

```
from langchain.retrievers.document_compressors import LLMChainExtractor
compressor = LLMChainExtractor.from_llm(llm)
compressed_docs = compressor.compress_documents(docs, query="Your query")
```

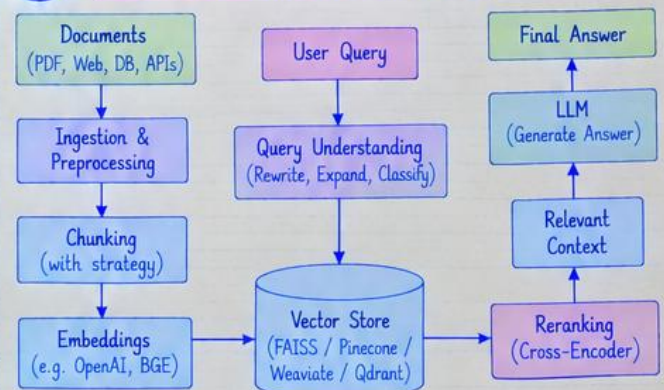
7 Evaluation Metrics

Metric	Description
Precision	% of retrieved docs that are relevant
Recall	% of relevant docs that are retrieved
MRR (Mean Reciprocal Rank)	Rank of first relevant document
nDCG	Ranking quality of retrieved results
Answer Relevance	How relevant the final answer is
Faithfulness	Whether the answer is grounded in context

Always evaluate RAG using both retrieval metrics and answer quality metrics.

"RAG is not just about finding information, but about delivering the right information."

2 Advanced RAG Architecture



4 Example: Hybrid Search (BM25 + Vector)

```
from langchain.retrievers import EnsembleRetriever
from langchain.retrievers import BM25Retriever
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings

# BM25 Retriever (keyword search)
bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = 5

# Vector Retriever (semantic search)
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(docs, embeddings)
vector_retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

# Ensemble Retriever (Hybrid Search)
ensemble = EnsembleRetriever(
    retrievers=[bm25_retriever, vector_retriever],
    weights=[0.5, 0.5]
)
results = ensemble.get_relevant_documents("What is RAG?")
```

Hybrid search combines keyword (BM25) and semantic (vector) search to get the best results.

6 Reranking with Cross-Encoder

- Initial retrieval gives top-k results, but they may not be perfectly relevant.
- Use a cross-encoder model (e.g. BERT, Cohere Rerank) to re-rank them.
- Provides better relevance by understanding query-document relationship.

Cross-encoder gives higher accuracy than simple similarity search.

```
from langchain.rerankers import CohereRerank
reranker = CohereRerank(model="rerank-english-v3", top_n=3)
reranked_docs = reranker.compress_documents(docs, query="Your query")
```

8 Key Takeaways

- ✓ Use advanced retrieval techniques (hybrid, rerank, filters).
- ✓ Apply context compression to fit more information.
- ✓ Choose the right vector store and embedding model.
- ✓ Evaluate your RAG system regularly.
- ✓ Use structured approaches for complex domains.
- ✓ RAG is a continuous improvement process.
- ✓ Combine RAG with tools and agents for more powerful apps.
- ✓ Focus on accuracy, reliability, and user experience.



"From information to action, with the help of Agents."

RAG with Agents in LangChain

Intelligent Applications that Retrieve, Reason and Act

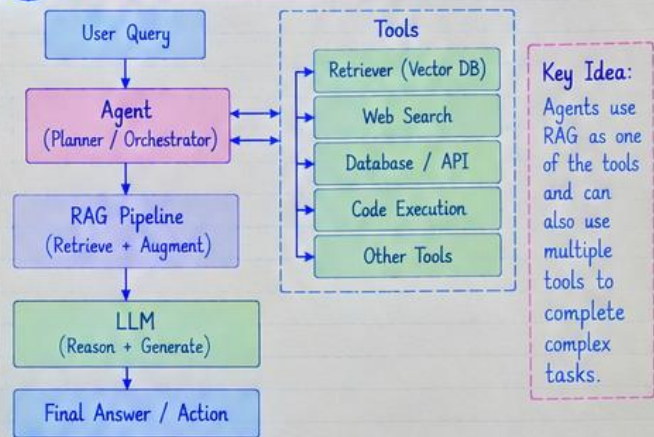
"RAG gives knowledge, Agents give action."

1 Why Combine RAG with Agents ?

- RAG helps LLMs get accurate, up-to-date information from external sources.
- Agents can plan, decide, and take actions using tools based on the retrieved information.
- Combining both enables LLMs to not just answer questions, but also perform real-world tasks (e.g., search, summarize, create reports, update databases).
- Useful for complex workflows where multiple steps, tools and external data are required.

"RAG gives the knowledge. Agents decide what to do with that knowledge."

2 High-Level Architecture



3 Components Used

Component	Library (LangChain)	Purpose
LLM	ChatOpenAI, ChatGrok, etc.	Generate and reason
Retriever	FAISS, Chroma, Pinecone	Retrieve relevant docs
Tools	@tool, Tool, API wrappers	Access external systems
Agent	initialize_agent, create_agent (LangGraph)	Plan and use tools
Memory (optional)	ConversationBufferMemory	Maintain conversation context
Chains (optional)	LCEL, RunnableSequence	Custom workflows

4 Example: RAG Agent with Search Tool

```
from langchain.agents import initialize_agent, AgentType, Tool
from langchain.tools import DuckDuckGoSearchRun
from langchain_community.vectorstores import FAISS
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain.chains import RetrievalQA

# Define search tool
search = DuckDuckGoSearchRun()
search_tool = Tool(
    name="search",
    func=search.run,
    description="Search the web for latest information"
)

# Initialize LLM
llm = ChatOpenAI(model="gpt-4o", temperature=0)
tools = [search_tool]
agent = initialize_agent(tools, llm, agent=AgentType.OPENAI_FUNCTIONS,
    verbose=True)

response = agent.invoke({"input": "Find latest updates on RAG and create a short summary."})
print(response["output"])
```

This agent can search the web, retrieve relevant information (using RAG if added) and generate a final answer.

5 Example: RAG Agent with Vector DB Tool

```
from langchain.tools import Tool
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.agents import initialize_agent, AgentType

# Load vector store
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(docs, embeddings)
retriever = vectorstore.as_retriever(search_type="similarity", k=4)

# Wrap retriever as a tool
retrieval_tool = Tool(
    name="document_search",
    func=lambda q: "\n\n".join([doc.page_content for doc in retriever.get_relevant_documents(q)]),
    description="Search internal documents for relevant information."
)

# Initialize agent
llm = ChatOpenAI(model="gpt-4o", temperature=0)
tools = [retrieval_tool]
agent = initialize_agent(tools, llm, agent=AgentType.OPENAI_FUNCTIONS)
result = agent.invoke({"input": "Explain Nykaa's return policy from internal docs."})
print(result["output"])
```

Replace docs with your company or domain documents (e.g., PDFs, Notion, etc.).

6 Real-World Use Cases

- Internal knowledge assistant (company docs, wikis, policies)
- Customer support automation (FAQs + live data)
- Research agent (search, summarize, analyze)
- Data analysis agent (query DB, run code, generate charts)
- Travel planner (search flights, hotels, create itinerary)
- Financial analysis (fetch data, analyze, generate report)
- Content creation (research + generate blog/report)
- Dev productivity (read codebase, search docs, suggest fixes)
- Multi-step enterprise workflows (RAG + APIs + DB + Actions)

7 Best Practices

- ✓ Use clear tool descriptions.
- ✓ Use the right retrieval strategy (similarity, hybrid, rerank).
- ✓ Combine multiple tools for complex tasks.
- ✓ Manage memory for multi-turn conversations.
- ✓ Limit tool access for safety and security.
- ✓ Evaluate agent performance (accuracy, latency, cost).
- ✓ Monitor and log agent actions.

8 Key Takeaways

- ✓ RAG provides factual, up-to-date information.
- ✓ Agents enable action, decision-making and multi-step workflows.
- ✓ LangChain makes it easy to combine RAG, tools and agents.
- ✓ You can build powerful real-world applications beyond Q&A.
- ✓ Start simple, add tools gradually and iterate.
- ✓ RAG + Agents = Knowledge + Action = Real Impact.

"Not just answers, but actions — that's the next step in AI."



"Learn
Build
Experiment
Repeat..."

LangChain - Final Notes & Summary

Key Takeaways, Resources and Next Steps

@abhi_techhub

"Better Tools
Better Products
Brighter Future."

1

What We Learned ?

- ✓ Memory helps LLMs remember and maintain context.
- ✓ Chains connect multiple components to handle complex tasks.
- ✓ Tools extend LLM capabilities to real-world applications.
- ✓ Retrievers (RAG) bring external knowledge into LLMs.
- ✓ Agents can plan, decide and use tools autonomously.
- ✓ LangChain provides simple abstractions for building all of this.
- ✓ Advanced techniques like hybrid search, reranking, and compression improve performance.
- ✓ Evaluation metrics help measure the quality of our system.
- ✓ LangChain is continuously evolving with new integrations and features.

2

When to Use What ?

Use Case / Problem	Recommended Approach
Remember past conversation	ConversationBufferMemory / SummaryMemory
Handle long conversations	SummaryMemory / VectorStoreMemory
Use external knowledge (docs)	RAG (Retrievers + Vector Store)
Perform multiple steps	Chains (Sequential / Router / etc.)
Need real-time data (web, APIs)	Tools (e.g. SerpAPI, custom tools)
Complex decision making	Agents (ReAct, OpenAI Functions)
Improve retrieval quality	Hybrid Search + Reranking
Reduce token usage	Context Compression
Domain specific application	Custom Chains + Tools + Memory

3

Best Practices

- ✓ Use the right memory type for your use case.
- ✓ Keep prompts clear and structured.
- ✓ Break complex tasks into smaller chains.
- ✓ Use tools wisely with proper descriptions.
- ✓ Provide relevant context through RAG.
- ✓ Evaluate your system with real-world data.
- ✓ Handle errors and edge cases gracefully.
- ✓ Monitor cost, latency and token usage.
- ✓ Keep the conversation relevant and avoid context pollution.
- ✓ Continuously experiment and iterate.

4

Common Mistakes to Avoid

- ✗ Using large context without compression.
- ✗ Choosing the wrong memory type.
- ✗ Poorly written prompts.
- ✗ Not handling API errors and rate limits.
- ✗ Using too many tools (confusing the agent).
- ✗ Relying only on keyword search (no semantic search).
- ✗ Ignoring evaluation and testing.
- ✗ Not optimizing for cost and performance.
- ✗ Forgetting to sanitize sensitive user data.
- ✗ Expecting perfect answers from LLMs (they can hallucinate).

5

Real-World Application Ideas

- 📄 Document Q&A Assistant (PDF, Notion, etc.)
- 💬 Customer Support Chatbot (e-commerce, SaaS)
- 🔍 Research Assistant (search + summarize)
- 📁 Code Assistant (use tools like GitHub, StackOverflow)
- 📧 Email/Calendar Assistant (use APIs)
- 📊 Data Analysis Assistant (CSV, SQL, dashboards)
- 🏥 Medical Information Assistant (domain specific)
- ⚖️ Legal Document Assistant
- 💡 Personal Knowledge Assistant (notes, bookmarks, etc.)

6

Learning Resources

- 📖 LangChain Official Docs → <https://python.langchain.com>
- 📖 LangChain GitHub → <https://github.com/langchain-ai/langchain>
- 📺 LangChain Tutorials (YouTube) → LangChain, freeCodeCamp, TechWithTim
- 📄 Hugging Face → <https://huggingface.co>
- ⚙️ OpenAI Documentation → <https://platform.openai.com/docs>
- ⚙️ Pinecone → <https://www.pinecone.io>
- 📖 FAISS → <https://github.com/facebookresearch/faiss>
- 📖 Weaviate → <https://weaviate.io>
- 🎥 ChromaDB → <https://www.trychroma.com>
- 📄 LangChain Community → <https://discord.gg/langchain>

7

Next Steps for You

- ✓ Build a small RAG project (e.g., PDF Q&A).
- ✓ Experiment with different memory types.
- ✓ Try building an agent with real tools (weather, web search).
- ✓ Work on a domain-specific project (e.g., resume analyzer).
- ✓ Deploy your application (Streamlit / FastAPI).
- ✓ Evaluate and improve based on user feedback.
- ✓ Keep exploring new LangChain features and integrations.

8

Final Thought

"LangChain is not just a library,
it's a bridge between LLMs and the real world.
The more you experiment, the more possibilities
you'll discover."

Build
Experiment
Learn
Grow 😊

"Small steps with LangChain today,
can build powerful AI products tomorrow."